

The Last Two Instructions

A forensic analysis of an unmatched function in the *Diddy Kong Racing* decompilation, and what it reveals about the IDO 5.3 code generator

Diddy Kong Racing decompilation project — research report, July 2026. Revised after the final construction session.
Artifact archive: `menu_research/` in the project repository.

ABSTRACT. The matching decompilation of *Diddy Kong Racing* (Rare, 1997) reproduces the retail ROM byte-for-byte except for a small number of functions. The hardest of these, the track-selection routine `func_8008FF1C` (`menu.c`), differs from the shipped binary by exactly two instructions — a single register name (`v1` vs. `t2`) in a load and its consuming branch. We report a multi-campaign investigation that (1) reverse-engineered the register-allocation machinery of the IDO 5.3 code generator (`ugen`) to a predictive, trace-verified model; (2) demonstrated with a patched compiler that plausible plain C produces the shipped bytes exactly if one internal code-generation decision is flipped; (3) bounded the source search along seven swept dimensions, including a finite enumeration of the code generator's expression dispatch for the target-compatible holder frame; (4) falsified every obtainable alternative toolchain, including the pressed retail IDO 5.3 CD and SGI's own freeware re-release, which we show to be byte-identical to the community toolchain for every compiler pass; (5) established by corpus statistics over another Rare title built with the byte-identical compiler that the anomalous pattern occurs nowhere else in Rare's shipped output, while its "collapsed" counterpart occurs routinely, including in a matched function with the identical source topology; and (6) showed by cross-revision analysis of all five retail ROM variants that the anomaly is bit-stable across the game's entire release history, surviving at least one genuine recompilation. The surviving explanations reduce to two: a persistent, undistributed toolchain difference confined to the development machine that built this game, or a deterministic source construct outside every family enumerated here. We assess the combined residual probability of a conventionally recoverable source-level match at under two percent, publish the complete negative-result ledger (~600 scored source variants, twelve toolchain combinations, seven binary lineages), and derive several results of general use to matching-decompilation projects.

1. Background

A *matching decompilation* reconstructs C source that, compiled with the original toolchain and settings, reproduces a shipped binary exactly. *Diddy Kong Racing* was built with SGI's IDO 5.3 compiler suite, a four-pass pipeline: `cfe` (C front end → ucode IR), `uopt` (global optimizer and register allocator for named variables), `ugen` (code generator, which invents all scratch registers), and `as1` (assembler: delay-slot filling, copy elimination). Matching therefore requires reproducing not just semantics but every bookkeeping choice of this pipeline.

The function at issue renders the track-selection menu. The project's best reconstruction compiles to 371 instructions of which 369 match the retail ROM bit-for-bit. The remaining two:

reconstruction:	<code>lh</code>	<code>v1,0(s1)</code>	<code>...</code>	<code>beq</code>	<code>v1,at,<else></code>
retail ROM:	<code>lh</code>	<code>t2,0(s1)</code>	<code>...</code>	<code>beq</code>	<code>t2,at,<else></code>

The site reads a 16-bit track identifier from a global array, stores an unrelated pointer, and branches on a comparison with `-1`; in the ROM, the store sits in the branch delay slot. The community had independently reached the same two-instruction residual (decomp.me scratch L6qKV, 99.69%), so this wall predates our work.

2. Methods

Scoring. Candidates are compiled as the full menu.c translation unit with the project's canonical toolchain and compared against the reference object. Two metrics are used: a mnemonic+register comparison for search guidance, and a strict full-instruction-word comparison with relocation fields masked (verifier tools/tryfw.py), required to be zero — together with the project's whole-ROM build verification — for any match claim. The distinction matters: the loose metric under-counted by one relocation-affected word in early work, a defect found and fixed during independent audit (§10).

Glass-box instrumentation. We used the community's static recompilation of the IDO binaries (decompals/ido-static-recomp), which translates the original IRIX MIPS executables instruction-for-instruction into native code with all 542 ugen procedures named. We added environment-gated logging and behavioral patches; in every case the modified binary was verified to produce byte-identical output with the gates disabled before any measurement was trusted. Pre-assembly listings (cc -K) and ucode listings (-Wo,-l...) provided intermediate-stage evidence.

Search campaigns. Approximately 600 source variants were constructed and scored across all campaigns (including ~58,000 iterations of the community's decomp-permuter in earlier sessions), organized into named families with a machine-readable ledger. Two independent audits reviewed the claims; both audits' corrections are incorporated and disclosed (§10).

3. A predictive model of the IDO 5.3 code generator's scratch allocation

The following model was read from the recompiled ugen source, confirmed by event tracing (267/267 allocation events in the final validation), and verified predictively. To our knowledge none of it was previously documented.

1. **FIFO free-queue.** Expression temporaries come from a queue over the MIPS t-registers {\$8-\$15, \$24, \$25}, seeded [14,15,24,25,8,9,10,11,12,13]. Allocation (get_free_reg) pops the head; a value's death re-appends its register to the tail. Register choice for scratch values is therefore a deterministic function of allocation history, not liveness alone.
2. **Pair cycle.** 64-bit values take even-aligned register pairs from a monotonic cycle (12,13)→(14,15)→(24,25)→(8,9)→(10,11); an alignment skip re-queues the skipped odd register.
3. **64-bit low-word additions** receive their destination from the free-queue like any temporary; an apparent “in-place” update arises when the operand's register has just been freed to the queue head — an emergent effect, confirmed at the dispatch site (hint = none → get_free_reg).
4. **The load-store collapse.** When a statement stores a leaf memory load into a register-resident variable, the statement handler passes the variable's register down as a destination hint and the load lands in it directly — no queue pop occurs. This single decision separates the reconstruction from the ROM.
5. **Dead-code interplay.** The optimizer deletes register-only dead statements before the code generator runs; dead *memory reads* survive to code generation (expression-table read protection), where they can consume queue slots before the assembler deletes them. The project's long-standing “load-bearing dead statement” (temp = (temp = load) with temp declared s16) works exactly this way: it burns one queue slot (§10) and emits nothing.
6. **The fixed point.** Given one premise — that the branch-feeding load is a queue pop (§10) — the queue model reproduces every scratch register in the second half of the function mechanically.

The ROM's register pattern is the queue's fixed point.

4. Sufficiency: one compiler bit away from the natural source

We patched the recompiled ugen (environment-gated; byte-identical when disabled) to drop the destination hint for this site's exact signature. Under that one-bit change, the plain, trick-free source

```
selectedTrack = gTrackSelectIDs[trackY][trackX];
cur->hubName = levelName;
if (selectedTrack != -1) { ... }
```

compiles to the shipped 371 instructions **exactly** (the same source scores 133 instruction differences under the stock compiler). The intermediate listing shows the predicted emission verbatim: `lh $10; move $3,$10` (assembler-dissolved); the store; the branch — and the queue fixed point then produces the entire downstream register pattern. This establishes *sufficiency*: this plausible plain source produces the shipped bytes under one changed code-generation decision. It does not establish uniqueness or historical provenance, a distinction the audits insisted on and we adopt.

5. Bounding the C source space

Could any C construct make the *stock* compiler defer the collapse at zero cost? Seven dimensions were audited, each by sweep or by reading the deciding code rather than by spelling permutation alone:

Dimension	Method	Outcome
RHS wrapper space (every ucode operator that can wrap the load inside the store statement)	Finite enumeration of the code generator's 152-opcode expression dispatch + empirical probes	Exactly one zero-cost deferring shape exists (an identity integer conversion) and the front end provably deletes it before code generation (byte-identical-object proof). Every surviving conversion costs a register pair, live shifts, or FPU traffic; every other operator either collapses or emits real code.
The “in-place add” residual of the community's 64-bit-mask idiom	Reading the destination-selection dispatch	The needs of the three sub-goals (pool-temp load; aligned downstream block; fresh add destination) are pairwise satisfiable but jointly contradictory through one queue state.
Register-web recoloring (forcing the holder to t2 in the optimizer)	Reading the coloring code + stacking experiments	A register is forbidden only by a <i>surviving</i> interfering live range; all zero-code levers are invisible to coloring (stacking 1–6 dead-statement burners leaves the color invariant), and real interference emits code (394 diffs observed).
Scope, lifetime, declaration order, dead-variable reuse	Sweep (~48 cells)	All collapse to v1. Declaration reordering is provably allocation-neutral.
Control-flow topology (switch, goto diamonds, store commoning, inversions, short-circuit forms)	Sweep	All negative; label-bearing forms perturb allocation globally.
Loop induction structure	Ten-form matrix	See §5.1 — partial success with a decisive incompatibility.
Aggregates, bitfields, external-declaration and prototype forms	Sweep (18+ forms)	Every form preserving the exact signed load still collapses; every collapse-dodging form costs stack traffic or allocation rotation.

5.1 The loop-induction discovery

The raw-condition form (branching on the array element directly) always produced the correct pool-temp load but emitted the pointer store *before* the load, because the optimizer could not prove the load and the store disjoint: the store goes through a loop-carried pointer whose symbol provenance is destroyed by strength reduction. We found that two loop topologies — fully direct indexing, and an explicit linear index `arr[k]` — cause the optimizer to retain the symbol at the point of use, emit a load/store disjointness directive (`.noalias`), and produce the **exact target microstructure** (load; compare; branch; store in the delay slot) under the completely stock toolchain — the first C-level reproduction of the shipped instruction order. The incompatibility: provenance retention requires a surviving symbolic index (~7 extra instructions), while the ROM uses a bare pointer induction, whose provenance the optimizer erases. The shipped compiler granted disjointness *for the bare pointer induction*; the canonical toolchain categorically does not. This yields a second single-decision route to the target, in the optimizer's alias analysis rather than the code generator's collapse.

6. Falsifying the alternative-toolchain hypotheses

Uniform behavioral variants are ROM-inconsistent. Under any code generator whose rule defers collapse for a class of sites containing ours, a direct load-into-web-register is unproducible *from any source* at sites in that class (deferral emits a pool load, and the assembler renames toward pool registers,

never into web registers). The matched portion of menu.c alone contains 67 such sites (20 with the exact narrowest signature). No uniform variant of the collapse rule is consistent with the shipped ROM, independent of source assumptions.

Every obtainable binary is the same binary. An exhaustive hunt across seven IDO/MIPSpro lineages on multiple archives established: the IDO 5.3 ugen is a *single build* (md5 9fcff937...) — identical across the community toolchain, the pressed retail “IRIS Development Option 5.3” CD (obtained and extracted twice, independently), and SGI's September 1997 freeware re-release; SGI's complete 5.3 patch record (398 patch manifests across both patch CDs) contains **no ugen patch whatsoever**; IDO 5.1/5.2 produce byte-identical output at this site, and IDO 6.0/7.1/7.4.4 collapse *more* aggressively while breaking 52 currently-matched functions. A twelve-cell Cartesian matrix over {cfe, uopt, as1} × {retail, patch builds} left the function byte-invariant in every cell, and collateral damage across the matrix **pins the game's actual toolchain**: retail cfe, retail 5.3MR uopt, and the community-circulated as1 build.

7. Statistical evidence from another Rare title

GoldenEye 007 is a complete matched IDO 5.3 Rare decompilation whose repository vendors the **original IRIX ugen binary — byte-identical to ours**. Across its 345 code objects:

- The anomalous pattern (signed halfword load into a pool temporary, surviving an unrelated store, feeding a compare-with-−1) occurs **zero** times.
- Corpus-wide, of the 186 temporary registers feeding a −1 comparison anywhere in the game, **none** is produced by a signed halfword load.
- The *collapsed* counterpart occurs 26 times — including one matched C function with the identical source topology to our site (16-bit structure field into a local; intervening stores; compare with −1) which ships collapsed, exactly as the canonical compiler behaves. Adapting its topology into our function reproduces our 133-difference collapse basin, as the model predicts.

An audit of Banjo-Kazooie's fully matched source (712 files) found no site with the relevant structure at all. On present evidence the DKR site is a **singleton across all measurable Rare output**: one occurrence of a pattern the same compiler binary demonstrably resolves the other way everywhere else, including in Rare's own matched code.

8. Cross-revision determinism

All five retail variants were screened with a calibrated byte-pattern search (one hit, zero false positives on the reference ROM):

ROM	Site form	Function vs. US Rev 1
USA v1.0	lh t2 (anomalous)	identical modulo relink (35 address-class diffs, 0 structural)
USA Rev 1	lh t2	reference
Europe v1.0	lh t2	identical modulo relink (35 / 0)
Europe Rev 1	lh t2	identical modulo relink (27 / 0)
Japan	lh t2	identical modulo relink (50 / 0)

The function is bit-stable across the game's entire release history: every difference in every revision is a link-address artifact (jump targets and high/low address immediates); no register, opcode, or structural

difference exists anywhere in its 371 instructions. Crucially, in the Japanese build, structurally changed code sits within ~1 KB of the function — inside its own translation unit's territory — so menu.c was genuinely recompiled for that release *and reproduced the anomaly exactly*. This falsifies the one-off build-accident hypothesis: whatever produced these two instructions was deterministic and persistent in the game's build environment for over a year.

9. Synthesis: what can still be true

The evidence now excludes: (i) every enumerated C source family under the canonical toolchain (§5); (ii) every uniform behavioral variant of the code generator (§6); (iii) every binary obtainable from public distribution, including the pressed retail media (§6); (iv) a one-off build accident (§8); and (v) any systematic Rare-wide toolchain difference (§7). Two explanations survive, both consistent with all observations:

H1 (favored): a persistent, undistributed toolchain difference on the DKR build machine. For example, an SGI support hot-fix or locally modified compiler pass installed on the machine that built this game — but not on the machines that built GoldenEye or Banjo-Kazooie — for the project's lifetime. Two concrete one-decision mechanisms would each produce the bytes from natural source: the code generator's collapse hint (§4) or the optimizer's alias tag for pointer inductions (§5.1). Nothing in public circulation can test this; Rare or Nintendo archival material could.

H2: a deterministic C construct outside every family enumerated here. The audits correctly note that family enumeration cannot cover all of C; the code-generator dispatch enumeration is complete for one statement frame, and the topology dimensions are swept rather than proven. We estimate the residual at under one to two percent, informed by a same-project precedent in which a comparably confident impossibility verdict was later overturned by an untried source topology (see §10).

A weakened object-reuse variant of H1 (a compiled object carried across builds) is consistent with the US/EU relink-only diffs but is disfavored by the Japanese recompilation evidence.

10. Error correction and audit history

In the spirit of adversarial collaboration we disclose the claims that did not survive:

- An early claim that the dispatch enumeration “covers all of C by construction” was an overreach; it is a lemma for one statement frame. Corrected after independent audit.
- An early blanket claim that no C respelling restores store provenance was true only under a fixed loop structure; whole-loop restructures do restore it (§5.1). Corrected by sweep.
- The guidance metric ignored relocation-field differences; conclusions survived a strict re-check, and the strict verifier is now the required gate.
- A sister-function campaign in this same project (the racer physics routine) had earlier been declared “unmatchable for any source” on a cost-model argument; it was subsequently solved by an untried source topology. That precedent sets the humility bar for §9's probability estimates, and motivated the dispatch-level (rather than family-level) closures used here.

11. Results of general use to decompilation projects

- **The ugen scratch-allocation model** (§3): FIFO free-queue, pair cycle, collapse rule, and dead-code interplay apply to every IDO 5.3 title (Nintendo 64 era, including all Rare decompilations). The “load-bearing dead statement” folklore is now mechanistically explained.
- **The glass-box method**: environment-gated instrumentation and behavioral patching of statically recompiled compiler binaries, with byte-fidelity gates. Any “why did the compiler pick this register” question is now directly observable.
- **Toolchain archaeology**: the pressed IDO 5.3 CD, SGI's freeware tardist, and the community binaries are pass-for-pass byte-identical for the compiler proper; no SGI 5.3 patch ever shipped a ugen; three distinct 5.3 as1 builds circulate (pressed/freeware, community, and patchSG0001118); a working IDO 5.1 ugen recompilation is provided (relocated-libc fix).
- **The ROM-anchored falsification pattern**: testing a hypothesized compiler variant source-independently by exhibiting sites in *matched* code that the variant could not produce from any source. Stronger than collateral counting, and cheap.
- **A strict, relocation-aware verifier** for match claims, and the demonstration that mnemonic-register metrics can under-count.

12. Reproducibility

The project repository's `menu_research/` archive contains: the scoring harness and strict verifier; 264 durable source variants plus generated construction scripts representing more than 48,000 overlapping late-stage cases; a machine-readable score ledger; the compiler instrumentation patch (with pristine-source diff); free-queue event traces; and the individual campaign reports with the commands, hashes, and labels needed to re-derive each claim. All compiler modifications are environment-gated and byte-fidelity-verified off. The five retail ROM analyses require legally owned dumps; we publish offsets, patterns, and hashes only.

13. Construction-session addendum

A skeptical follow-up session attacked the three places where this paper's original closure language was soft. First, it enumerated the missing retained-direct predecessors: address-only seeds, direct pointer-difference round trips, and a real direct store followed by an equal pointer assignment. None transfers no-alias state without target-visible address work. A flat continuous direct induction does reach the exact local target window, correcting the earlier overbroad claim that direct access always costs seven extra instructions, but flattening changes the retail nested-loop induction and register-allocation graph.

Second, the follow-up separated the real 32-bit index from semantically live 64-bit pair activity across producer, call-boundary, carrier, entry-pointer, and shift/subtract/shift-stage families. The strongest new state is 371 instructions with three differences at indices 106, 107, and 109; no family reached zero. Third, boundary tests for the non-alias producer confirmed that `const`, block scope, and parameter reuse remain ordinary collapsed-holder forms. Alias pragmas, `restrict`, hidden driver flags, and global static linkage also fail to alter the decision.

Corrected standing. The two-difference result is a strongly audited mechanism-level stopping point, not a theorem over every possible C program. Further broad syntax, mask, declaration-order, and loop-form sweeps revisit characterized compiler states. The source investigation should reopen only for a proposal that predicts a new normalized optimizer/code-generator state while preserving all 371 words, or for new historical build evidence. Exact scope and scores are in `report_construction_endgame.md` and `POSTMORTEM.md`.

14. Conclusion

Two instructions separate the *Diddy Kong Racing* decompilation from a perfect menu.c. They are no longer locally mysterious: two internal compiler decisions are independently sufficient to produce the shipped window, and the stock compiler's principal source routes are mechanistically characterized. Those oracle interventions do not identify the historical source or compiler. Short of a genuinely new normalized source topology or Rare's own build environment surfacing, the reconstruction at two instructions — with this dossier attached — is the correct and honest practical stopping point. The investigation itself has returned a compiler-internals dividend worth more to the community than the two register fields it set out to recover.

15. Postscript: the whole-ROM diagnostic demonstration

After this document was first assembled, a final experiment closed the loop at the strongest possible standard. The alias decision was localized to the optimizer's base-disjointness query (`f_base_noalias`), and the anomalous site's query was found to carry a descriptor signature that is a **singleton in descriptor space** across the entire ROM: no principled category predicate (kind-based, symbol-class, provenance) covers it without collateral, but the conjunction of two site-specific descriptor fields fires for exactly one query in the whole game. Under an environment-gated optimizer carrying that single grant, **the complete retail US v80 ROM was rebuilt byte-for-byte from the plain, natural C source for the anomalous function** (SHA-1 equal to retail; the project's full verification passing), with the gate disabled reproducing the stock toolchain exactly at whole-ROM scale. Sister-title forensics on two further Rare games supplied by legally-owned dumps (Jet Force Gemini, Mickey's Speedway USA — both fingerprinted as IDO 5.3-class output) found zero additional instances, extending the singleton to four games; and a memory-state sensitivity sweep of the recompiled code generator (13 poison patterns over heap and stack) proved the decision memory-independent, resolving the recompilation project's own “UB stack reads” annotation as an unrelated register-liveness artifact.

The diagnostic state of the mystery is therefore: plausible plain source plus exactly one target-derived alias grant can reproduce the shipped ROM — demonstrable, reproducible, and bit-exactly reenacted — while the committed reconstruction under the strictly stock toolchain remains at two instructions. The gated build is an oracle, not an acceptable stock-toolchain match and not evidence that the historical development machine carried that exact site-keyed rule. Both states are published with this dossier because they separate sufficiency from historical attribution.

External references: `decompals/ido-static-recomp` (static recompilation of IDO); `n64decomp/ido` (uopt decompilation); the `n64decomp` GoldenEye and Banjo-Kazooie projects; `decomp.me` scratch L6qKV; `jrra.zone` SGI CD archive; the SGI freeware mirror at `ftp.zx.net.nz`. This document describes clean-room compiler forensics; no leaked or proprietary material was used.